

Next Fit Voice

Technical Project Notes & Setup Guide

Technical Handoff & Learning Documentation

Prepared for: Next Fit Studio

Prepared by: Vantix Automation Solutions

Document Version: 1.0

Date: September 2026

TABLE OF CONTENTS

1. WHAT THIS PROJECT IS

- In Simple Language
- Who It Is Designed For
- What the Voice Agent Does
- External Services Involved

2. ARCHITECTURE OVERVIEW

- The Big Picture
- Arrow-by-Arrow Explanation
- Local Development Mode (Different Flow)

3. CALL LIFECYCLE

- Complete Sequence — Production Phone Call
- Local Development Flow (Text Chat)

4. FILE-BY-FILE PROJECT MAP

- Directory Tree
- Important Files Explained

5. THE VOICE AGENT

- Agent Initialization
- Agent Personality
- Conversation Phases
- Active Intent Detection
- Correction Handling
- Clarification Handling
- Model Configuration
- Greeting
- Call Termination
- Error Behavior

6. AGENT PROMPTS

- Where Prompts Live
- The Three Production Prompts
- Dynamic Prompt Construction
- How to Safely Modify Prompts

7. TOOLS / FUNCTIONS

- Important: This Is NOT a Traditional Agent

What the Backend Does (Pseudo-Tools)

The Lead Extraction Flow (Closest to "Tool Calling")

Deterministic Responses (Production Only)

8. BUSINESS LOGIC

What Is Implemented

Not Currently Implemented

Possible Future Extensions

9. ENVIRONMENT VARIABLES

Complete Reference

Where to Get Each Value

Example .env File (DO NOT use real keys)

Cloudflare Worker Secrets

10. SETUP GUIDE

Prerequisites

Step 1: Clone the Repository

Step 2: Create Environment File

Step 3: Set Up the Local Backend

Step 4: Run the Local Backend

Step 5: Set Up the Frontend (Optional)

Step 6: Test Locally

Step 7: Run Tests

Why Each Step Is Needed

11. GITHUB HANDOFF

Setting Up Your Own GitHub Repository

12. DEVELOPMENT WORKFLOW

Where to Modify Different Parts

What Should NOT Be Hardcoded

13. TESTING

Automated Tests

Manual Test Checklist

14. DEPLOYMENT

Production Architecture

Deployment Commands

Production Configuration

Required Production Secrets

Local vs Production Differences

Exotel Configuration

15. TROUBLESHOOTING

Agent Does Not Start

Call Does Not Connect

No Audio Response

LLM Failures

Frontend Issues

16. SECURITY

Current Security Status

Security Recommendations

What .gitignore Protects

17. CURRENT STATUS

WORKING / IMPLEMENTED

PARTIALLY IMPLEMENTED

NOT IMPLEMENTED

FUTURE IMPROVEMENTS

18. LEARNING GUIDE

LEVEL 1 — Understand the Project

LEVEL 2 — Understand the Agent

LEVEL 3 — Understand the Voice Pipeline

LEVEL 4 — Understand Lead Qualification

LEVEL 5 — Modify Behavior

LEVEL 6 — Add New Features

19. CHEAT SHEET

20. CODE QUALITY / TECHNICAL DEBT

CRITICAL

HIGH

MEDIUM

LOW

21. FUTURE ROADMAP

PHASE 1 — Stabilization

PHASE 2 — Better Reliability

PHASE 3 — More Business Capabilities

PHASE 4 — Analytics / Monitoring

PHASE 5 — Production Scaling

QUICK REFERENCE

Key URLs

Key Commands

NEXT FIT VOICE — PROJECT NOTES

Complete Technical Handoff & Teaching Document

This document explains everything about the NextFit AI Voice Receptionist project — what it is, how it works, where everything lives, and how to continue developing it.

1. WHAT THIS PROJECT IS

In Simple Language

NextFit Voice is an **AI receptionist that answers phone calls** for a gym called **NextFit** in Pune, India.

When someone calls the gym, instead of a human receptionist picking up, this AI: - Greets the caller naturally - Understands what they want (membership, personal training, trial, etc.) - Asks smart questions to qualify them as a lead - Collects their contact details - Scores how interested/ready they are (0-100) - Recommends what to do next (hot lead → hand off to human, etc.)

Who It Is Designed For

- **NextFit Studio** — a gym in Pune, India
- **The caller** — someone interested in joining the gym or inquiring about services
- **The gym owner/manager** — who wants to capture every lead automatically

What the Voice Agent Does

When a real person calls the gym phone number:

1. They hear a friendly, natural-sounding AI voice
2. The AI introduces itself as the NextFit receptionist
3. It asks what the caller is interested in
4. It discovers their fitness goals, current situation, and problems
5. It qualifies them based on how ready they are to join
6. It collects their name and phone number
7. It either schedules a callback or connects them to a human
8. The gym owner gets a full lead profile with a qualification score

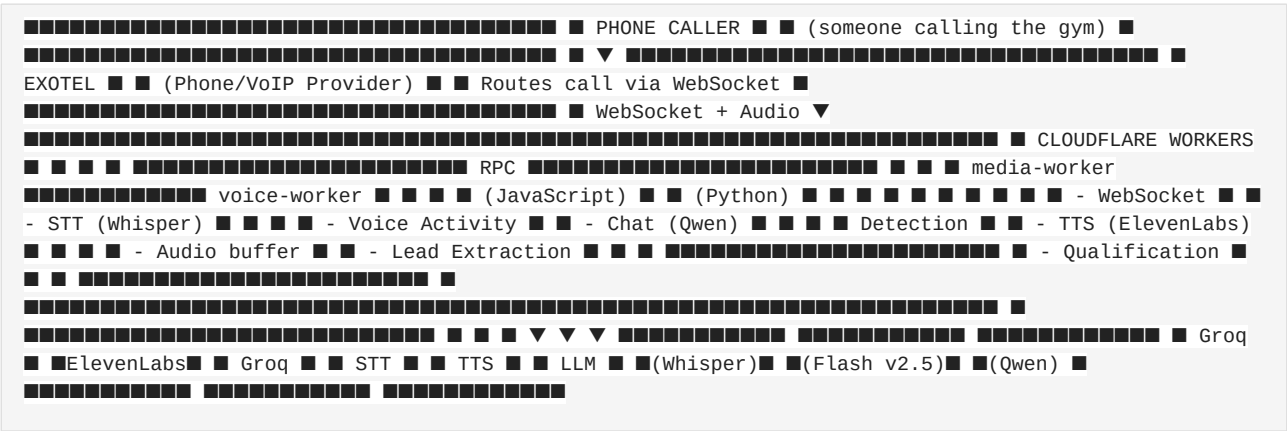
External Services Involved

Service	What It Does	Why It's Needed
---------	--------------	-----------------

Exotel	Phone/VoIP provider	Routes phone calls to the AI
Cloudflare Workers	Hosting platform	Runs the AI application
Groq	AI inference (LLM + STT)	Processes language and speech
ElevenLabs	Text-to-speech	Converts AI responses to natural voice

2. ARCHITECTURE OVERVIEW

The Big Picture



Arrow-by-Arrow Explanation

Arrow	What Happens
Caller → Exotel	The person dials the gym's phone number, which routes to Exotel (VoIP)
Exotel → media-worker	Exotel opens a WebSocket and streams audio chunks (8kHz, mono, 16-bit)
media-worker (VAD)	Accumulates audio, detects when the caller stops speaking (750ms silence)
media-worker → voice-worker	Sends the complete utterance via Cloudflare RPC (<code>process_utterance</code>)
voice-worker → Groq STT	Converts audio to text using <code>whisper-large-v3-turbo</code>
voice-worker → Groq LLM	Sends text + conversation context to <code>qwen/qwen3.6-27b</code> for response
voice-worker → ElevenLabs	Converts the AI's text response to speech

voice-worker → media-worker	Returns audio bytes + updated conversation state
media-worker → Exotel	Sends audio back through the WebSocket in chunks
Exotel → Caller	The caller hears the AI's response through their phone

Local Development Mode (Different Flow)

```
React Frontend (port 5173) ■ ■ POST /chat, POST /tts ▼ Backend FastAPI (port 8787 or 8000) ■ ■■■ Groq API (Chat + Lead Extraction) ■■■ ElevenLabs API (TTS)
```

In local dev, the React frontend acts as a text chat interface with optional browser-based speech. The phone integration is only active in production.

3. CALL LIFECYCLE

Complete Sequence — Production Phone Call

```
1. CALLER DIALS Someone calls the NextFit phone number. 2. EXOTEL ROUTES Exotel (VoIP provider) receives the call and connects it via WebSocket to the media-worker at api.vantixlab.info/media. 3. WEBSOCKET CONNECTS media-worker/src/index.js accepts the WebSocket. Exotel sends "connected" event. Creates a fresh connection state (audio buffer, conversation state). 4. STREAM STARTS Exotel sends "start" event with streamSid and callSid. The worker begins listening for audio. 5. AUDIO STREAMS IN Exotel sends "media" events with base64-encoded PCM audio (8kHz, mono). Worker decodes and accumulates audio chunks. Worker checks RMS (volume) every 5 packets – this is Voice Activity Detection. 6. SILENCE DETECTED When RMS drops below 500 for 750ms, the worker assumes the caller finished speaking. It concatenates all audio chunks into one utterance. 7. RPC CALL media-worker calls PYTHON_AI.process_utterance() via Cloudflare Service Binding. Sends: raw PCM audio bytes + current conversation state. 8. STT – Speech to Text voice_rpc.py converts PCM to WAV format. Calls Groq's Whisper API (whisper-large-v3-turbo) with the audio. Gets back transcribed text. 9. CHAT PROCESSING The transcribed text goes through the full chat pipeline: a. Clarification detection ("what did you say?") b. Correction detection ("actually, make it Saturday") c. Intent detection (membership, personal training, etc.) d. Topic interruption detection ("wait, before that...") e. Lead extraction (if message contains lead-relevant info) f. Phase progression (greeting → discovery → qualification → action → closing) g. System prompt construction with all context h. LLM generates response (Qwen qwen3.6-27b, temp 0.75) i. Response cleaning (strip thinking tags, meta prefixes) j. Second lead extraction (for any new info in the response) k. Qualification scoring l. Handoff eligibility check 10. TTS – Text to Speech The cleaned response text is sent to ElevenLabs. ElevenLabs generates audio in pcm_8000 format (telephony compatible). 11. AUDIO RETURNS voice_rpc.py returns: audio bytes + updated conversation state. media-worker receives the audio. 12. AUDIO CHUNKED & SENT media-worker splits audio into 6400-byte chunks. Sends each chunk to Exotel via WebSocket. Pads to 320-byte boundaries for proper telephony framing. 13. CALLER HEARS RESPONSE The audio plays through the caller's phone. The conversation continues from step 5. 14. CALL ENDS When the caller hangs up, Exotel sends a "stop" event. The WebSocket closes. Conversation state is lost (it's in-memory only).
```

Local Development Flow (Text Chat)

```
1. User types in React frontend 2. Frontend sends POST /chat with message 3. Backend processes (same pipeline as step 9 above) 4. Backend returns JSON: {response, lead, score, classification, reasons} 5. Frontend displays response + lead dashboard 6. Frontend optionally reads response aloud (browser TTS)
```


4. FILE-BY-FILE PROJECT MAP

Directory Tree

nextfit-ai-demo/ ■ ■■■ .env ← API keys (NEVER commit this) ■■■ .gitignore ← Git ignore rules ■■■
package.json ← Empty placeholder for npm root ■■■ requirements.txt ← Python dependencies ■ ■■■
backend/ ← LOCAL DEVELOPMENT server ■ ■■■ __init__.py ■ ■■■ main.py ← FastAPI app (1868 lines) –
THE core file ■ ■■■ models.py ← Pydantic data models (LeadProfile, etc.) ■ ■■■ conversation.py ←
Conversation state definitions ■ ■■■ context.py ← Conversation flow intelligence (600 lines) ■ ■■■
qualification.py ← Lead scoring engine (464 lines) ■ ■■■ prompts.py ← AI system prompts (798 lines)
■ ■■■ nextfit_config.py ← Business configuration ■ ■■■ test_qualification.py ← Manual test script ■
■■■ requirements.txt ← Pinned backend dependencies ■ ■■■ cloudflare-worker/ ← PRODUCTION deployment
■ ■■■ .dev.vars ← CF Worker dev secrets (NEVER commit) ■ ■■■ wrangler.jsonc ← Cloudflare Worker
config ■ ■■■ pyproject.toml ← Python project config ■ ■■■ package.json ← Wrangler npm scripts ■
■■■ src/ ■ ■■■ __init__.py ■ ■■■ entry.py ← Worker entrypoint (ASGI bridge) ■ ■■■ main.py ←
Async production app (2096 lines) ■ ■■■ models.py ← Same as backend/models.py ■ ■■■
conversation.py ← Same as backend/conversation.py ■ ■■■ context.py ← Improved context management ■
■ ■■■ qualification.py ← Same as backend/qualification.py ■ ■■■ prompts.py ← Three prompts (1426
lines) ■ ■■■ nextfit_config.py ← Same as backend/nextfit_config.py ■ ■■■ voice_rpc.py ← Voice
pipeline: STT → Chat → TTS ■ ■■■ test_qualification.py ■ ■■■ test_context.py ← Comprehensive
unit tests ■ ■■■ media-worker/ ← WebSocket handler for phone calls ■ ■■■ wrangler.jsonc ■ ■■■ src/
■ ■■■ index.js ← VAD, audio streaming, Exotel integration ■ ■■■ frontend/ ← React demo dashboard
■■■ package.json ← React 19, Vite 8 ■ ■■■ vite.config.js ■ ■■■ src/ ■ ■■■ main.jsx ← React entry ■
■■■ App.jsx ← Full chat + lead panel UI (1112 lines) ■ ■■■ App.css ← Component styles ■ ■■■
index.css ← Global styles ■ ■■■ public/ ■ ■■■ favicon.svg

Important Files Explained

backend/main.py — The Local Development Server

What	Detail
Purpose	FastAPI application that handles chat, TTS, and health endpoints
Size	1868 lines
Key endpoints	POST /chat, POST /tts, POST /reset, GET /health
LLM	Groq API, model qwen/qwen3.6-27b, temperature 0.65 for chat, 0.1 for extraction
TTS	ElevenLabs, model eleven_flash_v2_5, format mp3_22050_32
State	Single global ConversationState() — not thread-safe, dev-only
What would break without it	Everything — this IS the backend

cloudflare-worker/src/main.py — The Production Server

What	Detail
------	--------

Purpose	Async version of the backend, optimized for Cloudflare Workers
Size	2096 lines
Key differences	Uses httpx instead of SDKs, ContextVar for per-request state, conditional lead extraction, deterministic responses for simple greetings, rate limit handling
Chat prompt	Uses NEXTFIT_CHAT_PROMPT (voice-optimized, 426 lines)
Max tokens	220 for chat, 320 for extraction (shorter for voice)
What would break without it	No production deployment, no phone calls

`cloudflare-worker/src/voice_rpc.py` — The Voice Pipeline

What	Detail
Purpose	Bridges phone audio to the AI — STT → Chat → TTS
Size	232 lines
Key function	<code>process_utterance()</code> — receives PCM audio, returns audio response
STT	Groq Whisper <code>whisper-large-v3-turbo</code>
Audio format	8000Hz, 16-bit, mono (telephony standard)
What would break without it	No phone call capability — voice calls would not work

`cloudflare-worker/media-worker/src/index.js` — The Phone Bridge

What	Detail
Purpose	Handles WebSocket connection from Exotel, manages audio streaming
Size	470 lines (JavaScript)
Key features	Voice Activity Detection (VAD), audio buffering, chunked audio sending
VAD	RMS-based, threshold 500, silence detection 750ms
What would break without it	No phone calls — this is the WebSocket handler that Exotel connects to

backend/prompts.py / cloudflare-worker/src/prompts.py — AI Instructions

What	Detail
Purpose	Defines the AI's personality, behavior rules, and conversation guidelines
backend	2 prompts: NEXTFIT_SYSTEM_PROMPT (506 lines), LEAD_EXTRACTION_PROMPT (292 lines)
cloudflare-worker	3 prompts: adds NEXTFIT_CHAT_PROMPT (426 lines, voice-optimized)
What would break without it	The AI would have no personality, no rules, no idea how to behave

backend/context.py / cloudflare-worker/src/context.py — Conversation Intelligence

What	Detail
Purpose	Manages conversation state, intent detection, corrections, topic interruptions
Size	~600 lines
Key functions	<code>detect_active_intent()</code> , <code>is_correction()</code> , <code>process_correction()</code> , <code>is_clarification_request()</code> , <code>handle_clarification()</code> , <code>build_conversation_context()</code>
What would break without it	No conversation memory, no correction handling, no topic management

backend/qualification.py / cloudflare-worker/src/qualification.py — Lead Scoring

What	Detail
Purpose	Deterministic scoring engine — calculates lead quality 0-100
Size	464 lines
Key function	<code>calculate_qualification(lead)</code> → returns score + classification
Classifications	HOT (≥80), QUALIFIED (≥65), NURTURE (≥45), INFORMATION (≥20), LOW (<20)

What would break without it	No lead scoring — the system wouldn't know which leads to prioritize
-----------------------------	--

backend/models.py — Data Definitions

What	Detail
Purpose	Defines all data structures (Pydantic models)
Key classes	LeadProfile, LeadUpdate, QualificationResult
What would break without it	No data validation, no structured lead data

backend/conversation.py — State Definitions

What	Detail
Purpose	Defines ConversationState and ConversationMessage
Key fields	messages, lead, conversation_phase, active_intent, turn_count
What would break without it	No conversation state management

backend/nextfit_config.py — Business Info

What	Detail
Purpose	Static business configuration for NextFit
Content	Business name: "NextFit", Location: "Pune", Services: Gym Membership, Personal Training, Trial, Fitness Programs
Rules	Never invent prices, discounts, timings, or policies
What would break without it	AI wouldn't know basic business information

frontend/src/App.jsx — Demo Dashboard

What	Detail
Purpose	React UI for testing the AI via text chat with live lead qualification display
Size	1112 lines

Features	Text chat, browser speech recognition, lead score visualization, classification badges
API URL	Hardcoded to <code>http://127.0.0.1:8787</code>
What would break without it	No visual way to test the AI — would need to use API directly

5. THE VOICE AGENT

Agent Initialization

The agent does NOT use a traditional agent framework (like LangChain, AutoGen, etc.).

Instead, it's a **custom pipeline**:

1. Receive audio from phone
2. Convert speech → text (STT)
3. Build system prompt dynamically
4. Call LLM with system prompt + conversation history
5. Clean the response
6. Convert text → speech (TTS)
7. Return audio

There is no "agent loop" or "tool calling" in the traditional sense. The "tools" are handled by the backend code, not by the LLM.

Agent Personality

Defined in `prompts.py`, the AI receptionist is:

- **Friendly** — warm, approachable, not robotic
- **Confident** — knows about fitness but doesn't lecture
- **Relaxed** — casual tone, like "a good gym-bro who actually knows fitness"
- **Human-like** — never reveals it's AI unless directly asked
- **Concise** — 1-3 sentences per response (for phone calls)
- **Helpful first** — answers questions before qualifying

Conversation Phases

The agent progresses through phases:

```
GREETING → DISCOVERY → QUALIFICATION → ACTION → CLOSING
```

Phase	What Happens
Greeting	Hello, introduce self, ask what brings them in
Discovery	Understand goals, current situation, problems
Qualification	Ask about experience, timeline, availability
Action	Recommend service, suggest next step, collect contact
Closing	Confirm details, say goodbye

Active Intent Detection

The system detects what the caller wants:

Intent	Keywords
membership	"membership", "join", "sign up", "enroll"
personal_training	"personal training", "trainer", "one-on-one", "PT"
trial	"trial", "try", "free session", "demo"
class	"class", "yoga", "Zumba", "spin"
callback	"call me back", "callback", "reach me"
existing_member	"already member", "current member", "renew"
general_information	"info", "details", "timings", "price"

Correction Handling

If the caller corrects themselves ("actually, make it Saturday"), the system: 1. Detects the correction pattern 2. Updates the relevant field (date, time, preference) 3. Acknowledges the change naturally 4. Doesn't treat it as new information

Clarification Handling

If the caller says "what?", "huh?", "can you repeat?": 1. The system detects it locally 2. Repeats the last AI response 3. Limits to 2 consecutive clarifications before falling through to LLM

Model Configuration

Setting	Local Backend	Production (CF Worker)
---------	---------------	------------------------

Chat Model	qwen/qwen3.6-27b	qwen/qwen3.6-27b
Chat Temperature	0.65	0.75
Chat Max Tokens	400	220
Extraction Model	qwen/qwen3.6-27b	qwen/qwen3.6-27b
Extraction Temperature	0.1	0.1
Extraction Max Tokens	1200	320
STT Model	N/A (local only)	whisper-large-v3-turbo
TTS Model	eleven_flash_v2_5	eleven_flash_v2_5
TTS Output	mp3_22050_32	pcm_8000 (for phone)

Greeting

The greeting is NOT hardcoded — it's generated by the LLM based on the prompt instructions. The prompt tells the AI to: - Be natural and warm - Introduce itself as the NextFit receptionist - Ask what brings the caller in - Keep it short (1-2 sentences)

Call Termination

The call ends when: - The caller hangs up (Exotel sends "stop" event) - The WebSocket closes - No explicit "goodbye" logic exists — conversation state is simply lost

Error Behavior

Error	What Happens
STT fails	Returns None — the system logs it but has no recovery
LLM rate limit	Returns fallback: "I'm sorry, I'm having a technical issue. Can I take your number and have someone call you back?"
TTS fails	Returns empty bytes — the caller hears silence
Lead extraction fails	Skips extraction, continues with existing lead data

6. AGENT PROMPTS

Where Prompts Live

File	Location
Local backend	backend/prompts.py (2 prompts, 798 lines)
Production	cloudflare-worker/src/prompts.py (3 prompts, 1426 lines)

The Three Production Prompts

1. NEXTFIT_CHAT_PROMPT (426 lines) — THE ACTIVE ONE

This is the prompt actually used in production. It's voice-call optimized.

Section	What It Controls
Personality	Friendly, confident, casual tone
Most Important Rule	UNDERSTAND FIRST, HELP SECOND, QUALIFY THIRD
Opening Guidelines	How to greet callers
Customer Questions Come First	Always answer their question before qualifying
Natural Discovery	How to ask about goals, problems, experience
Conversational Priority	Don't sound like a form — sound like a human
Memory Rules	Don't ask what you already know
Handoff Rules	When to offer connecting to a human
Contact Collection	When/how to ask for name and phone
Tone Guidelines	Short sentences, casual, not corporate
Response Length	1-3 sentences (for phone)

2. LEAD_EXTRACTION_PROMPT (285 lines)

Instructs the LLM to extract structured lead data from the conversation.

Section	What It Controls
Allowed Values	Defines valid options for experience, timeline, etc.
JSON Format	Specifies the exact output structure
Critical Rules	Never guess, never invent, don't force missing info

False Positive Prevention	Date mentioned in a question ≠ joining intent
---------------------------	---

3. NEXTFIT_SYSTEM_PROMPT (631 lines)

The original text-based prompt (used in local backend). Similar to NEXTFIT_CHAT_PROMPT but: - Longer responses allowed - More detailed fitness knowledge - Less focused on phone call constraints

Dynamic Prompt Construction

The actual prompt sent to the LLM is built dynamically in main.py:

```
System Prompt (base personality + rules) + Business Info (from nextfit_config.py) + Conversation Context (from context.py) + Known Information (already collected about the caller) + Qualification Status (what's been scored) + Contact Status (name/phone collection progress) + Output Rules (no thinking tags, no reasoning)
```

How to Safely Modify Prompts

- 1. **Change personality/tone** → Edit the "Personality" section in the prompt
- 2. **Change business info** → Edit nextfit_config.py (NOT the prompt)
- 3. **Change conversation rules** → Edit the relevant section in the prompt
- 4. **Change what gets extracted** → Edit LEAD_EXTRACTION_PROMPT
- 5. **Always test** after changes — the prompt is sensitive to wording

7. TOOLS / FUNCTIONS

Important: This Is NOT a Traditional Agent

This system does NOT use function/tool calling in the way ChatGPT or LangChain agents do.

The LLM does NOT decide to call tools. Instead:

- **The backend code** handles all tool-like behavior
- **The LLM** only generates text responses
- **The backend** extracts structured data from the LLM's output

What the Backend Does (Pseudo-Tools)

Backend Function	Purpose	When Used	External System
Lead Extraction	Extracts structured data from conversation	Every turn (conditionally in production)	Groq LLM
Intent Detection	Determines what the caller wants	Every turn	Local code (keyword matching)

Correction Handling	Processes corrections to previously collected data	When correction detected	Local code
Clarification Handling	Repeats last response when caller didn't hear	When clarification detected	Local code
Topic Interruption	Handles subject changes, saves pending topic	When interruption detected	Local code
Qualification Scoring	Scores lead 0-100 and classifies	After every turn	Local code (deterministic)
Phone Number Normalization	Formats extracted phone numbers	After extraction	Local code
Experience Inference	Infers fitness experience from conversation	After extraction	Local code
Timeline Inference	Infers joining timeline from conversation	After extraction	Local code

The Lead Extraction Flow (Closest to "Tool Calling")

```
User message ↓ should_extract_lead(message) – checks if extraction is needed ↓ (only if lead-relevant)
Call Groq API with LEAD_EXTRACTION_PROMPT + conversation ↓ LLM returns JSON with lead data ↓ Backend
cleans/normalizes the JSON ↓ Merges with existing LeadProfile ↓ Qualification scoring runs on updated
lead
```

Deterministic Responses (Production Only)

The production system handles simple inputs WITHOUT calling the LLM:

Input	Response
"hi", "hello", "hey"	Greeting (predefined)
"how are you"	"I'm good, thanks! How can I help you today?"

This saves API costs and reduces latency.

8. BUSINESS LOGIC

What Is Implemented

Customer Handling: - Detects caller intent (membership, personal training, trial, etc.) - Asks discovery questions about goals and current situation - Understands fitness experience level - Handles objections naturally - Collects name and phone number

Lead Qualification: - 9 scoring components (0-100 total) - 5 classification levels: HOT, QUALIFIED, NURTURE, INFORMATION, LOW - Recommended actions per classification - Handoff eligibility (score $\geq$ 65 + willing to continue)

Conversation Intelligence: - Correction handling (fix previously given info) - Clarification handling (repeat when not heard) - Topic interruption (handle subject changes) - Duplicate question prevention - Phase progression (greeting → discovery → qualification → action → closing)

Service Information: - Gym Membership - Personal Training - Trial Sessions - Fitness Programs - Location: Pune

Not Currently Implemented

- **No actual booking/scheduling** — the AI talks about appointments but doesn't book them
- **No class schedule lookup** — no database of actual class times
- **No pricing information** — the rules explicitly say "do not invent prices"
- **No trainer profiles** — no database of trainers
- **No payment processing** — purely conversational
- **No CRM integration** — leads are not saved anywhere persistent
- **No call recording** — no storage of conversations
- **No analytics dashboard** — no monitoring of call metrics
- **No multi-location support** — hardcoded to Pune
- **No multi-language support** — English only
- **No after-hours handling** — no time-based behavior

Possible Future Extensions

- Integration with a CRM (HubSpot, Salesforce) to store leads
- Actual class schedule database
- Pricing information integration
- Call recording and transcription storage
- Analytics dashboard for call metrics
- Multi-language support
- WhatsApp integration
- SMS follow-up automation
- Appointment booking API integration
- Integration with gym management software

9. ENVIRONMENT VARIABLES

Complete Reference

Variable	Required?	Purpose	Example Format	Where Used
GROQ_API_KEY	Yes	API key for Groq (LLM + STT)	gsk_...	backend/main.py, cloudflare-worker/src/main.py, cloudflare-worker/src/voice_r
ELEVENLABS_API_KEY	Yes	API key for ElevenLabs (TTS)	sk_...	backend/main.py, cloudflare-worker/src/main.py
ELEVENLABS_VOICE_ID	Yes	Voice ID for ElevenLabs TTS	alphanumeric string	backend/main.py, cloudflare-worker/src/main.py

Where to Get Each Value

Variable	Source
GROQ_API_KEY	Sign up at https://console.groq.com → API Keys → Create
ELEVENLABS_API_KEY	Sign up at https://elevenlabs.io → Profile → API Keys
ELEVENLABS_VOICE_ID	In ElevenLabs, go to VoiceLab → Select a voice → Copy Voice ID

Example .env File (DO NOT use real keys)

Create a file called `.env` in the project root:

```
GROQ_API_KEY=YOUR_GROQ_API_KEY_HERE ELEVENLABS_API_KEY=YOUR_ELEVENLABS_API_KEY_HERE
ELEVENLABS_VOICE_ID=YOUR_ELEVENLABS_VOICE_ID_HERE
```

Cloudflare Worker Secrets

For production deployment, these same three variables must be set in Cloudflare:

```
# Using wrangler npx wrangler secret put GROQ_API_KEY npx wrangler secret put ELEVENLABS_API_KEY npx
wrangler secret put ELEVENLABS_VOICE_ID
```

Or set them via the Cloudflare Dashboard → Workers & Pages → nextfit-ai-voice → Settings → Variables.

10. SETUP GUIDE

Prerequisites

Before starting, you need: - [] A computer (Windows, Mac, or Linux) - [] Git installed - [] Python 3.12+ installed - [] Node.js 18+ installed - [] A Groq API key - [] An ElevenLabs API key - [] An ElevenLabs voice ID

Step 1: Clone the Repository

```
git clone https://github.com/YOUR_USERNAME/nextfit-ai-demo.git cd nextfit-ai-demo
```

Step 2: Create Environment File

Create a file called `.env` in the project root:

```
GROQ_API_KEY=YOUR_GROQ_API_KEY_HERE ELEVENLABS_API_KEY=YOUR_ELEVENLABS_API_KEY_HERE  
ELEVENLABS_VOICE_ID=YOUR_ELEVENLABS_VOICE_ID_HERE
```

Replace the placeholder values with your actual API keys.

Step 3: Set Up the Local Backend

```
# Create virtual environment python -m venv venv # Activate it (Windows) venv\Scripts\activate #  
Activate it (Mac/Linux) source venv/bin/activate # Install dependencies pip install -r requirements.txt
```

Step 4: Run the Local Backend

```
# From the project root, with venv activated cd backend python -m uvicorn main:app --reload --host  
127.0.0.1 --port 8787
```

The backend starts at `http://127.0.0.1:8787`.

Step 5: Set Up the Frontend (Optional)

```
# In a new terminal cd frontend npm install npm run dev
```

The frontend starts at `http://localhost:5173`.

Step 6: Test Locally

1. Open `http://localhost:5173` in your browser
2. Type a message in the chat
3. You should see the AI respond
4. The lead panel on the right shows qualification data

Step 7: Run Tests

```
# From project root cd cloudflare-worker/src python test_context.py python test_qualification.py
```

Why Each Step Is Needed

Step	Why
Clone	Gets the code onto your machine
.env file	Provides API keys the app needs to call Groq and ElevenLabs
venv + pip install	Installs Python dependencies without conflicting with other projects
uvicorn	Starts the Python web server
npm install + dev	Installs React dependencies and starts the dev server
Tests	Verifies the qualification and context logic works correctly

11. GITHUB HANDOFF

Setting Up Your Own GitHub Repository

Step 1: Create a New Repository on GitHub

1. Go to <https://github.com>
2. Click the + icon → **New repository**
3. Name it: `nextfit-ai-demo` (or whatever you prefer)
4. Choose **Private** (recommended — contains API key references)
5. Do NOT initialize with README, .gitignore, or license
6. Click **Create repository**

Step 2: Connect Your Local Project to the New Repo

```
# From the project directory git remote -v # This shows the current remote (probably the developer's
repo) # Change the remote to your new repo git remote set-url origin
https://github.com/YOUR_USERNAME/YOUR_REPO_NAME.git # Verify the change git remote -v
```

Step 3: Push Your Code

```
# Make sure you're on the main branch git branch # Stage all files (except what .gitignore protects)
git add . # Commit git commit -m "Initial commit: NextFit AI Voice Receptionist" # Push to your repo
git push -u origin main
```

What .gitignore Protects

The `.gitignore` file prevents these from being committed:

Pattern	What It Is	Why It's Protected
<code>venv/</code>	Python virtual environment	Large, machine-specific
<code>.venv/</code>	Another Python venv pattern	Same as above
<code>.env</code>	API keys file	SECRETS — must never be committed
<code>__pycache__/</code>	Python bytecode cache	Generated, not needed
<code>*.pyc</code>	Python compiled files	Generated, not needed

Important Security Notes

- The `.env` file is in `.gitignore` and should NOT be committed
- If you accidentally committed it before, you need to remove it from git history
- The `cloudflare-worker/.dev.vars` file also contains secrets and should NOT be committed
- **If you see API keys in your git history, rotate them immediately**

Recommended Branch Structure

```
main ← Production-ready code ■■■ dev ← Your development branch (optional)
```

For a simple project like this, working directly on `main` is fine.

Day-to-Day Git Workflow

```
# Pull latest changes git pull origin main # Make your changes... # Stage and commit git add . git commit -m "Description of what you changed" # Push git push origin main
```

12. DEVELOPMENT WORKFLOW

Where to Modify Different Parts

I want to change...	Go to...
Agent personality	<code>cloudflare-worker/src/prompts.py</code> → <code>NEXTFIT_CHAT_PROMPT</code>
Agent greeting behavior	<code>cloudflare-worker/src/prompts.py</code> → "Opening Guidelines" section
Business information	<code>backend/nextfit_config.py</code> or <code>cloudflare-worker/src/nextfit_config.py</code>

What questions the AI asks	<code>cloudflare-worker/src/prompts.py</code> → "Natural Discovery" section
Lead scoring rules	<code>backend/qualification.py</code> or <code>cloudflare-worker/src/qualification.py</code>
Lead classifications	<code>backend/qualification.py</code> → <code>calculate_qualification()</code>
Conversation flow phases	<code>backend/context.py</code> → <code>update_conversation_phase()</code>
Intent detection keywords	<code>backend/context.py</code> → <code>detect_active_intent()</code>
Correction handling	<code>backend/context.py</code> → <code>process_correction()</code>
AI model used	<code>backend/main.py</code> → <code>model="qwen/qwen3.6-27b"</code>
TTS voice	Change <code>ELEVENLABS_VOICE_ID</code> in <code>.env</code>
TTS model	<code>backend/main.py</code> → <code>model_id="eleven_flash_v2_5"</code>
Temperature (creativity)	<code>backend/main.py</code> → <code>temperature=0.65</code>
Response length limit	<code>backend/main.py</code> → <code>max_tokens=400</code>
VAD sensitivity	<code>cloudflare-worker/media-worker/src/index.js</code> → <code>VAD_THRESHOLD = 500</code>
Silence timeout	<code>cloudflare-worker/media-worker/src/index.js</code> → <code>SILENCE_MS = 750</code>
Phone number normalization	<code>backend/main.py</code> → <code>normalize_phone_number()</code>
CORS allowed origins	<code>backend/main.py</code> or <code>cloudflare-worker/src/main.py</code> → <code>CORSMiddleware</code>
Frontend API URL	<code>frontend/src/App.jsx</code> → <code>API_URL = "http://127.0.0.1:8787"</code>

What Should NOT Be Hardcoded

Don't hardcode...	Instead use...
API keys	Environment variables
Business hours	<code>nextfit_config.py</code> or database

Pricing	nextfit_config.py or database
Phone numbers	Configuration or database
Trainers	Configuration or database
Class schedules	Database

13. TESTING

Automated Tests

Context Tests (Comprehensive)

```
cd cloudflare-worker/src python test_context.py
```

Tests 11+ scenarios: - A: Intent detection for membership - B: Name/membership remembrance - C: Correction detection and intent change - D: Date correction ("Actually Saturday") - E: Time correction ("Actually make that 7") - F: Normal question not misclassified - G: Clarification detection and last response repeat - H: "That's not what I meant" handling - I: "I'll think about it" not confirmed - J: Callback intent detection - K: Data persistence after many turns - Phase transition tests - Message selection tests - Topic interrupt tests

Qualification Tests

```
cd backend python test_qualification.py
```

Tests a "perfect" lead scenario (should score as HOT).

Manual Test Checklist

Basic Conversation

- ☐ AI answers call / responds to chat input
- ☐ Greeting sounds natural
- ☐ AI understands normal speech/text
- ☐ AI answers business FAQ correctly
- ☐ AI doesn't invent prices or policies

Lead Qualification

- ☐ AI asks about fitness goals
- ☐ AI asks about current situation
- ☐ AI asks about experience level

- ☐ AI asks about timeline
- ☐ AI asks about availability
- ☐ Lead score updates correctly
- ☐ Classification changes appropriately

Correction Handling

- ☐ AI handles "actually, make it Saturday"
- ☐ AI handles "wait, I meant personal training"
- ☐ AI handles "no, I said next week"

Clarification Handling

- ☐ AI repeats itself when asked "what?"
- ☐ AI doesn't repeat more than 2 times

Edge Cases

- ☐ AI handles silence gracefully
- ☐ AI handles interruption ("wait, before that...")
- ☐ AI handles off-topic questions
- ☐ AI handles "I need to go"
- ☐ AI handles incomplete information
- ☐ AI doesn't crash on empty input

Technical

- ☐ Backend starts without errors
- ☐ Health endpoint returns OK
- ☐ Reset endpoint clears state
- ☐ Frontend connects to backend
- ☐ Lead panel displays data correctly

14. DEPLOYMENT

Production Architecture

Exotel (Phone Provider) ↓ WebSocket Cloudflare Worker: nextfit-ai-media (JavaScript) ↓ Service Binding
RPC Cloudflare Worker: nextfit-ai-voice (Python) ↓ HTTP calls Groq API (STT + LLM) + ElevenLabs API
(TTS)

Deployment Commands

```
# From cloudflare-worker directory cd cloudflare-worker # Install uv (Python package manager) if not installed # Windows: winget install astral-sh.uv # Mac: brew install uv # Install dependencies uv sync # Deploy the voice worker npm run deploy # This runs: uv run pywrangler deploy # Deploy the media worker cd media-worker npx wrangler deploy
```

Production Configuration

Setting	Value
Voice Worker Name	nextfit-ai-voice
Media Worker Name	nextfit-ai-media
Custom Domain	api.vantixlab.info
Python Version	3.12
Compatibility Date	2026-08-20

Required Production Secrets

Set these in Cloudflare Dashboard or via wrangler:

```
# For nextfit-ai-voice worker npx wrangler secret put GROQ_API_KEY npx wrangler secret put ELEVENLABS_API_KEY npx wrangler secret put ELEVENLABS_VOICE_ID
```

Local vs Production Differences

Aspect	Local Development	Production
Backend	FastAPI + uvicorn	Cloudflare Workers (Python)
Phone Calls	Not available	Exotel WebSocket
Frontend	React dev server	Not deployed (demo only)
State	Global variable	ContextVar per-request
LLM SDK	groq Python SDK	Direct httpx calls
TTS SDK	elevenlabs Python SDK	Direct httpx calls
CORS	localhost:5173	vantixlab.info domains
Max Tokens	400 (chat), 1200 (extract)	220 (chat), 320 (extract)
Deterministic Responses	No	Yes (for "hi", "hello")

Conditional Extraction	No (always extracts)	Yes (only when relevant)
------------------------	----------------------	--------------------------

Exotel Configuration

The phone provider (Exotel) must be configured to: 1. Forward incoming calls to the WebSocket endpoint 2. Point the media stream to `wss://api.vantixlab.info/media` 3. Use 8kHz, mono, 16-bit PCM audio format

This is configured in the Exotel dashboard, not in this codebase.

15. TROUBLESHOOTING

Agent Does Not Start

Symptom	Likely Cause	How to Check	How to Fix
<code>ModuleNotFoundError</code>	Dependencies not installed	Check venv is activated	Run <code>pip install -r requirements.txt</code>
<code>GROQ_API_KEY</code> is missing	<code>.env</code> file missing or empty	Check <code>.env</code> exists	Create <code>.env</code> with your API key
<code>Address already in use</code>	Port 8787 already running	Check for other processes	Kill the process or use different port

Call Does Not Connect

Symptom	Likely Cause	How to Check	How to Fix
Exotel shows "connection failed"	WebSocket endpoint wrong	Check Exotel config	Verify <code>wss://api.vantixlab.info/media</code>
WebSocket connects but no audio	Media worker not deployed	Check Cloudflare dashboard	Deploy media-worker
Audio is garbled	Wrong audio format	Check Exotel settings	Ensure 8kHz, mono, 16-bit PCM

No Audio Response

Symptom	Likely Cause	How to Check	How to Fix
Caller hears silence after speaking	STT failed	Check worker logs	Verify Groq API key

Caller hears silence after AI responds	TTS failed	Check worker logs	Verify ElevenLabs API key
Audio is too quiet	VAD threshold too high	Check <code>VAD_THRESHOLD</code>	Lower the threshold value

LLM Failures

Symptom	Likely Cause	How to Check	How to Fix
Rate limit errors	Too many requests	Check Groq dashboard	Implement queuing or wait
Gibberish responses	Temperature too high	Check <code>temperature</code> setting	Lower to 0.65
Thinking tags in response	Model outputting reasoning	Check <code>clean_spoken_response()</code>	Update cleaning regex
AI says "I don't know" to everything	Prompt too restrictive	Review prompt wording	Adjust system prompt

Frontend Issues

Symptom	Likely Cause	How to Check	How to Fix
"Cannot connect to API"	Backend not running	Check backend is on port 8787	Start backend
Blank screen	JavaScript error	Open browser console	Check for errors
Lead panel not updating	API response format changed	Check response structure	Verify ChatResponse model

16. SECURITY

Current Security Status

Area	Status	Notes
API Keys	In <code>.env</code> file	<code>.gitignore</code> protects from commit, but keys are on disk
No Authentication	All endpoints are open	Anyone can call <code>/chat</code> or <code>/tts</code>

No Rate Limiting	Server-side	Relies on Groq's rate limiting
No Input Validation	Minimal	Pydantic models provide basic validation
No HTTPS Locally	HTTP only	Production uses HTTPS via Cloudflare
PII Handling	None	Names and phones are stored in-memory only, lost on restart
No Database	In-memory only	No persistent storage of any customer data

Security Recommendations

Priority	Recommendation
CRITICAL	Rotate exposed API keys if they were ever committed to git
CRITICAL	Never commit <code>.env</code> or <code>.dev.vars</code> files
HIGH	Add API key authentication for <code>/chat</code> and <code>/tts</code> endpoints
HIGH	Add rate limiting to prevent abuse
MEDIUM	Add input length limits to prevent token exhaustion
MEDIUM	Log suspicious activity
LOW	Add CORS restrictions for production

What `.gitignore` Protects

```
venv/ .venv/ .env ← Contains API keys __pycache__/ *.pyc
```

Note: The `cloudflare-worker/.dev.vars` file also contains secrets. It may need to be added to `.gitignore` if not already covered.

17. CURRENT STATUS

WORKING / IMPLEMENTED

- FastAPI backend with chat, TTS, health, and reset endpoints

- ■ Lead extraction from conversation using LLM
- ■ Deterministic lead qualification scoring (0-100)
- ■ 5-level lead classification (HOT/QUALIFIED/NURTURE/INFORMATION/LOW)
- ■ Conversation state management (phases, intents, corrections, clarifications)
- ■ Intent detection (membership, personal training, trial, class, callback)
- ■ Correction handling (date, time, preference corrections)
- ■ Clarification handling (repeat last response)
- ■ Topic interruption and resume
- ■ Duplicate question prevention
- ■ Dynamic system prompt construction
- ■ Text cleaning (thinking tags, reasoning blocks, markdown)
- ■ Phone number normalization
- ■ Experience and timeline inference from conversation
- ■ Cloudflare Workers deployment (Python worker + JS media worker)
- ■ Exotel WebSocket integration for phone calls
- ■ Voice Activity Detection (VAD) for phone calls
- ■ Groq Whisper STT integration
- ■ ElevenLabs TTS integration
- ■ React demo dashboard with lead visualization
- ■ Browser-based speech recognition (demo)
- ■ Deterministic responses for simple greetings (production)
- ■ Conditional lead extraction (production)
- ■ Rate limit handling with fallback response (production)
- ■ Comprehensive context tests
- ■ Qualification scoring tests

PARTIALLY IMPLEMENTED

- ■■ Frontend is a demo only — not production-ready
- ■■ Code duplication between backend/ and cloudflare-worker/src/
- ■■ Production prompts differ from local prompts (drift)
- ■■ No persistent conversation storage

NOT IMPLEMENTED

- ■ No CRM integration (leads are not saved)
- ■ No actual appointment booking
- ■ No class schedule database

- ■ No pricing information
- ■ No call recording
- ■ No analytics/monitoring
- ■ No multi-language support
- ■ No WhatsApp integration
- ■ No SMS follow-up
- ■ No authentication/authorization
- ■ No database of any kind
- ■ No CI/CD pipeline

FUTURE IMPROVEMENTS

- Add persistent storage (database) for leads and conversations
- Integrate with a CRM (HubSpot, Salesforce)
- Add actual class schedule and pricing information
- Implement appointment booking
- Add call recording and transcription
- Build an analytics dashboard
- Add multi-language support
- Integrate WhatsApp channel
- Add SMS follow-up automation
- Implement user authentication
- Add rate limiting and abuse prevention
- Set up CI/CD pipeline

18. LEARNING GUIDE

LEVEL 1 — Understand the Project

Goal: Know what this project is and how it's organized.

Files to study: 1. `PROJECT_NOTES.md` (this document) — Sections 1-4 2. `backend/nextfit_config.py` — Basic business info 3. `backend/models.py` — Data structures 4. `backend/conversation.py` — State definitions

What to understand: - What the project does (AI phone receptionist for a gym) - The directory structure (backend, cloudflare-worker, frontend) - The data models (LeadProfile, ConversationState) - The business configuration

LEVEL 2 — Understand the Agent

Goal: Know how the AI conversation works.

Files to study: 1. `cloudflare-worker/src/prompts.py` — Read the `NEXTFIT_CHAT_PROMPT` 2. `cloudflare-worker/src/context.py` — Conversation intelligence 3. `cloudflare-worker/src/main.py` — The chat endpoint flow

What to understand: - The AI's personality and rules - How conversation phases work - How intent detection works - How the chat endpoint processes each message

LEVEL 3 — Understand the Voice Pipeline

Goal: Know how phone calls work.

Files to study: 1. `cloudflare-worker/media-worker/src/index.js` — WebSocket + VAD 2. `cloudflare-worker/src/voice_rpc.py` — STT → Chat → TTS 3. `cloudflare-worker/src/main.py` — Production chat flow

What to understand: - How Exotel connects via WebSocket - How audio is buffered and detected (VAD) - How speech is converted to text (STT) - How text is converted back to speech (TTS) - How conversation state flows through the pipeline

LEVEL 4 — Understand Lead Qualification

Goal: Know how leads are scored.

Files to study: 1. `backend/qualification.py` — The scoring engine 2. `backend/prompts.py` — The `LEAD_EXTRACTION_PROMPT`

What to understand: - How the LLM extracts structured data - How each scoring component works - How classification thresholds work - How handoff eligibility is determined

LEVEL 5 — Modify Behavior

Goal: Be able to change how the AI behaves.

Files to modify: 1. `cloudflare-worker/src/prompts.py` — Change personality, rules, questions 2. `backend/nextfit_config.py` — Change business information 3. `backend/qualification.py` — Change scoring rules 4. `backend/context.py` — Change conversation flow logic

What to understand: - How prompts affect AI behavior - How to add new intent types - How to modify scoring weights - How to change conversation phases

LEVEL 6 — Add New Features

Goal: Be able to add new capabilities.

Files to modify: 1. `backend/main.py` or `cloudflare-worker/src/main.py` — Add new endpoints 2. `cloudflare-worker/src/voice_rpc.py` — Modify voice pipeline 3. `cloudflare-worker/media-worker/src/index.js` — Modify WebSocket handling

What to understand: - How to add new API endpoints - How to integrate new external services - How to modify the voice pipeline - How Cloudflare Workers deployment works

19. CHEAT SHEET

I want to...	Go to...
Change agent personality	cloudflare-worker/src/prompts.py → NEXTFIT_CHAT_PROMPT "Personality" section
Change greeting	cloudflare-worker/src/prompts.py → "Opening Guidelines" section
Change what questions are asked	cloudflare-worker/src/prompts.py → "Natural Discovery" section
Change business information	backend/nextfit_config.py or cloudflare-worker/src/nextfit_config.py
Change AI model	backend/main.py line ~1635 or cloudflare-worker/src/main.py line ~1859
Change AI temperature	backend/main.py line ~1637 (0.65) or cloudflare-worker/src/main.py line ~1859 (0.75)
Change response length	backend/main.py line ~1638 (max_tokens=400) or cloudflare-worker/src/main.py line ~227 (CHAT_MAX_TOKENS=220)
Change voice	Change ELEVENLABS_VOICE_ID in .env or Cloudflare secrets
Change TTS model	backend/main.py line ~1825 or cloudflare-worker/src/main.py line ~382
Change lead scoring	backend/qualification.py → calculate_qualification()
Change classification thresholds	backend/qualification.py lines 414-441
Change intent detection	backend/context.py → detect_active_intent()
Change correction handling	backend/context.py → process_correction()
Change conversation phases	backend/context.py → update_conversation_phase()
Change VAD sensitivity	cloudflare-worker/media-worker/src/index.js line 7 (VAD_THRESHOLD=500)
Change silence timeout	cloudflare-worker/media-worker/src/index.js line 4 (SILENCE_MS=750)
Change CORS origins	cloudflare-worker/src/main.py lines 159-164

Change API credentials	<code>.env</code> (local) or Cloudflare secrets (production)
Change phone normalization	<code>backend/main.py</code> → <code>normalize_phone_number()</code>
Change frontend API URL	<code>frontend/src/App.jsx</code> line 4 (<code>API_URL</code>)
Change extraction logic	<code>backend/main.py</code> → <code>extract_lead_data()</code>
Change lead extraction prompt	<code>backend/prompts.py</code> → <code>LEAD_EXTRACTION_PROMPT</code>
Deploy to production	<code>cd cloudflare-worker && npm run deploy</code>
Run tests	<code>cd cloudflare-worker/src && python test_context.py</code>
Start local backend	<code>cd backend && python -m uvicorn main:app --reload --port 8787</code>
Start local frontend	<code>cd frontend && npm run dev</code>

20. CODE QUALITY / TECHNICAL DEBT

CRITICAL

Issue	Location	Impact
API keys may be in git history	<code>.env</code> , <code>.dev.vars</code>	Security risk — keys should be rotated
No authentication on endpoints	<code>backend/main.py</code> , <code>cloudflare-worker/src/main.py</code>	Anyone can use the API

HIGH

Issue	Location	Impact
Code duplication between <code>backend/</code> and <code>cloudflare-worker/src/</code>	<code>models.py</code> , <code>conversation.py</code> , <code>qualification.py</code> , <code>nextfit_config.py</code>	Changes must be made in two places
Code drift between local and production	<code>backend/main.py</code> vs <code>cloudflare-worker/src/main.py</code>	Production has improvements local doesn't
In-memory state only	All <code>ConversationState</code> usage	State lost on restart, not scalable

Single global state in local backend	backend/main.py global conversation_state	Not thread-safe, breaks with concurrent users
No error recovery for STT failure	cloudflare-worker/src/voice_rpc.py	Callers hear nothing if STT fails

MEDIUM

Issue	Location	Impact
Frontend API URL hardcoded	frontend/src/App.jsx line 4	Must change code for different environments
CORS includes development domains in production	cloudflare-worker/src/main.py	Minor security concern
test_qualification.py has wrong imports in cloudflare-worker	cloudflare-worker/src/test_qualification.py	Test won't run from that directory
Overlapping CSS files	frontend/src/App.css + index.css	Confusion about which to edit
No type hints in some functions	Various	Harder to maintain
No logging beyond console.log	cloudflare-worker/media-worker/src/index.ts	Harder to debug in production

LOW

Issue	Location	Impact
Empty root package.json	package.json	Confusing — appears unused
Duplicate dependency lists	Root requirements.txt + backend/requirements.txt	Must update both
No .env.example file	Project root	New developers don't know what variables are needed
No CI/CD pipeline	Project root	Manual deployment only
Version strings differ (0.7.1 vs 0.7.0)	backend/main.py vs cloudflare-worker/src/main.py	Minor inconsistency

21. FUTURE ROADMAP

PHASE 1 — Stabilization

- ☐ Rotate and secure all API keys
- ☐ Add `.env.example` file
- ☐ Add API key authentication
- ☐ Fix code duplication (shared models module)
- ☐ Add proper error handling for STT failures
- ☐ Add input validation
- ☐ Add logging

PHASE 2 — Better Reliability

- ☐ Add persistent storage (PostgreSQL or Redis) for conversation state
- ☐ Add call recording and transcription storage
- ☐ Add retry logic for external API calls
- ☐ Add health monitoring and alerting
- ☐ Add rate limiting

PHASE 3 — More Business Capabilities

- ☐ Add class schedule database
- ☐ Add pricing information
- ☐ Add trainer profiles
- ☐ Add appointment booking
- ☐ Add CRM integration (HubSpot/Salesforce)
- ☐ Add SMS follow-up automation

PHASE 4 — Analytics / Monitoring

- ☐ Add call analytics dashboard
- ☐ Add lead conversion tracking
- ☐ Add conversation quality metrics
- ☐ Add A/B testing for prompts
- ☐ Add conversation recording review

PHASE 5 — Production Scaling

- ☐ Add multi-location support
- ☐ Add multi-language support
- ☐ Add WhatsApp channel
- ☐ Add CI/CD pipeline

- [] Add load testing
- [] Add disaster recovery

QUICK REFERENCE

Key URLs

URL	Purpose
<code>http://127.0.0.1:8787</code>	Local backend
<code>http://localhost:5173</code>	Local frontend
<code>https://api.vantixlab.info</code>	Production API
<code>wss://api.vantixlab.info/media</code>	Production WebSocket (Exotel)
<code>https://console.groq.com</code>	Groq API dashboard
<code>https://elevenlabs.io</code>	ElevenLabs dashboard
<code>https://dash.exotel.com</code>	Exotel dashboard
<code>https://dash.cloudflare.com</code>	Cloudflare dashboard

Key Commands

```
# Local development python -m venv venv venv\Scripts\activate # Windows pip install -r requirements.txt
cd backend && python -m uvicorn main:app --reload --port 8787 cd frontend && npm run dev # Production
deployment cd cloudflare-worker npm run deploy # Deploys voice worker cd media-worker && npx wrangler
deploy # Deploys media worker # Testing cd cloudflare-worker/src python test_context.py python
test_qualification.py
```

This document was generated from a thorough inspection of the actual codebase. Every claim is based on code that exists in the repository. No features were invented or assumed.